

List in Python

21 September 2025 22:51

The data type list is an ordered sequence which is mutable and made up of one or more elements. A list is a very useful to group together elements of mixed data types. Elements of a list are enclosed in square brackets and are separated by comma. List indices starts from 0.

List of daily essentials:

```
daily_essentials = ['milk', 'water', 'bread', 'fruits', 'vegetables']
```

daily_essentials →

0	1	2	3	4
'milk'	'water'	'bread'	'fruits'	'vegetables'
-5	-4	-3	-2	-1

print(daily_essentials[2])
output: bread.

```
daily_essentials = ['milk', 'bread', 'tea', 'coffee', 'water', 'fruits']  
print(daily_essentials)  
print(daily_essentials[2])
```

Output:

```
['milk', 'bread', 'tea', 'coffee', 'water', 'fruits']  
tea
```

```
daily_essentials = ['milk', 'bread', 'tea', 'coffee', 'water', 'fruits']  
print(daily_essentials)  
print(daily_essentials[2])  
nums = [10, 12, 15, -9, 56, 61, 2, 19, 99]  
print(nums)  
float_nums = [2.3, 6.8, 8.3, -9.21, 9.99, 8.12]  
print(float_nums)  
print(float_nums[10])
```

Output:

↑ not a valid index

```
['milk', 'bread', 'tea', 'coffee', 'water', 'fruits']  
tea  
[10, 12, 15, -9, 56, 61, 2, 19, 99]  
[2.3, 6.8, 8.3, -9.21, 9.99, 8.12]  
Traceback (most recent call last):  
  File "c:\Users\bluej\Documents\Students\VaanyaSharmaPearsonMiddleSchool\  
Coding\Chapter4\Prog1.py", line 12, in <module>  
    print(float_nums[10])  
~~~~~  
IndexError: list index out of range
```

0 1 2 3 4 5
[2.3, 6.8, 8.3, -9.21, 9.99, 8.12]
-6 -5 -4 -3 -2 -1

```

daily_essentials = ['milk', 'bread', 'tea', 'coffee', 'water', 'fruits']
print(daily_essentials)
print(daily_essentials[2])
nums = [10, 12, 15, -9, 56, 61, 2, 19, 99]
print(nums)
float_nums = [2.3, 6.8, 8.3, -9.21, 9.99, 8.12]
print(float_nums)
# print(float_nums[10])
print(float_nums[-1]) #last element or value

```

index '0' means 1st element in the list.

index '-1' means last element in the list.

Output:

```

['milk', 'bread', 'tea', 'coffee', 'water', 'fruits']
tea
[10, 12, 15, -9, 56, 61, 2, 19, 99]
[2.3, 6.8, 8.3, -9.21, 9.99, 8.12]
8.12

```

Lists are Mutable

In Python, lists are mutable. It means that the contents of the list can be changed after it has been created.

```

colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
print(colours)
colours[1] = 'Purple'
print(colours)

```

Output:

```

['Red', 'Green', 'Blue', 'Orange', 'Pink']
['Red', 'Purple', 'Blue', 'Orange', 'Pink']

```

List Operations

The data type list allows manipulation of its content through various operations as shown below:

1. **Concatenation:** Python allows us to join two or more list using concatenation operator depicted by the symbol plus('+').

```

colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
print('colours =', colours)
colours[1] = 'Purple'
print('colours =', colours)
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
print('colours2 =', colours2)
moreColors = colours + colours2
print('moreColors =', moreColors)

```

After concatenation the actual lists will remain same. It will create a new list.

Output:

```

colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
colours = ['Red', 'Purple', 'Blue', 'Orange', 'Pink']
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
moreColors = ['Red', 'Purple', 'Blue', 'Orange', 'Pink', 'Lightgreen', 'PowderBlue', 'Cyan']

```

```

colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
print('colours =', colours)

```

```

colours[1] = 'Purple'
print('colours =', colours)
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
print('colours2 =', colours2)
moreColors = colours + colours2
print('moreColors =', moreColors)
print('colours =', colours)
print('colours2 =', colours2)
colours += colours2
print('Updated colours =', colours)

```

We can update the existing list by concatenation

Output:

```

colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
colours = ['Red', 'Purple', 'Blue', 'Orange', 'Pink']
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
moreColors = ['Red', 'Purple', 'Blue', 'Orange', 'Pink', 'Lightgreen', 'PowderBlue', 'Cyan']
colours = ['Red', 'Purple', 'Blue', 'Orange', 'Pink']
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
Updated colours = ['Red', 'Purple', 'Blue', 'Orange', 'Pink', 'Lightgreen', 'PowderBlue', 'Cyan']

```

Repetition: Python allow us to replicate a list using repetition operator depicted by symbol *.

```

list1 = ['Vannya']
print(list1)
list2 = list1*4
print(list2)
nums = [1, 2, 3, 4, 5]
rep_nums = nums * 3
print(nums)
print(rep_nums)

```

Output:

```

['Vannya']
['Vannya', 'Vannya', 'Vannya', 'Vannya']
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]

```

Membership: The membership operator 'in' checks if the element is present in the list and returns True else returns False.

```

colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
print('colours =', colours)
colours[1] = 'Purple'
print('colours =', colours)
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
print('colours2 =', colours2)
moreColors = colours + colours2
print(moreColors)
if 'Cyan' in moreColors:
    print('Cyan is present in the list')
else:
    print('cyan is not present in the list')
if 'Black' in moreColors:
    print('Yes')
else:
    print('No')

```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
colours = ['Red', 'Purple', 'Blue', 'Orange', 'Pink']
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
['Red', 'Purple', 'Blue', 'Orange', 'Pink', 'Lightgreen', 'PowderBlue', 'Cyan']
Cyan is present in the list
No
```

The `'not in'` operator returns `True` if the element is not present in the list, else it returns `False`.

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
print('colours =', colours)
colours[1] = 'Purple'
print('colours =', colours)
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
print('colours2 =', colours2)
moreColors = colours + colours2
print(moreColors)
if 'Cyan' not in moreColors:
    print('Cyan is not present in the list')
else:
    print('Cyan is present in the list')

if 'Black' not in moreColors:
    print('Yes')
else:
    print('No')
```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink']
colours = ['Red', 'Purple', 'Blue', 'Orange', 'Pink']
colours2 = ['Lightgreen', 'PowderBlue', 'Cyan']
['Red', 'Purple', 'Blue', 'Orange', 'Pink', 'Lightgreen', 'PowderBlue', 'Cyan']
Cyan is present in the list
Yes
```

Home work:

Python List Questions

1. Indexing Basics

What will be the output of the following code?

```
fruits = ['apple', 'banana', 'cherry']
print(fruits[1])
```

2. Negative Indexing

Given the list `colors = ['red', 'green', 'blue', 'yellow']`, what does `colors[-2]` return?

3. Mutability in Action

Consider the code below. What will be the final value of `numbers`?

```
numbers = [10, 20, 30]
numbers[1] = 99
```

4. Concatenation

What is the result of the following operation?

```
list1 = [1, 2]
list2 = [3, 4]
result = list1 + list2
```

5. Repetition

What does the following code output?

```
items = ['a', 'b']
print(items * 3)
```

6. Membership Operator - in

What will be printed?

```
pets = ['dog', 'cat', 'rabbit']
print('cat' in pets)
```

7. Membership Operator - not in

Evaluate the result:

```
nums = [1, 2, 3, 4]
print(5 not in nums)
```

8. IndexError Scenario

What happens when you run this code?

```
names = ['Alice', 'Bob']
print(names[5])
```

9. Combining Concepts

What will be the output of this code?

```
a = [1, 2]
b = a * 2 + [3]
print(b)
```

Slicing :

0 1 2 3 4 5 6 7

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
```

colours[2:6]
start → end-1

[start < end]
-6 < -2

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
subColoursList = colours[2:6]
print(subColoursList)
```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
```

```
subColoursList = colours[2:6]
print(subColoursList)
print(colours[2:20])
```

second index is out of range.

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
subColoursList = colours[2:6]
print(subColoursList)
print(colours[2:20])
print(colours[20:2])
```

← The first index > second index, results in an empty list.

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
[]
```

*[start : end]
↑ optional*

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
subColoursList = colours[2:6]
print(subColoursList)
print(colours[2:20])
print(colours[20:2])
print(colours[:5]) #start index is missing
print(colours[2:]) #end index is missing
print(colours[:]) #both indexes are missing
```

*compulsary
[start : end : step]
↑ optional*

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
[]
['Red', 'Green', 'Blue', 'Orange', 'Pink']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
subColoursList = colours[2:6]
print(subColoursList)
print(colours[2:20])
print(colours[20:2])
print(colours[:5]) #start index is missing
print(colours[2:]) #end index is missing
print(colours[:]) #both indexes are missing
print(colours[:2])
print(colours[2:6:2])
```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
[]
['Red', 'Green', 'Blue', 'Orange', 'Pink']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Blue', 'Pink', 'Magenta']
['Blue', 'Pink']
```

Negative indexing:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
subColoursList = colours[2:6]
print(subColoursList)
print(colours[2:20])
print(colours[20:2])
print(colours[:5]) #start index is missing
print(colours[2:]) #end index is missing
print(colours[:]) #both indexes are missing
print(colours[:2])
print(colours[2:6:2])
#Negative indexing
print('='*20)
print('Negative Indexing')
print('='*20)
print(colours[-6:-2])
```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
[]
['Red', 'Green', 'Blue', 'Orange', 'Pink']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Blue', 'Pink', 'Magenta']
['Blue', 'Pink']
=====
Negative Indexing
=====
['Blue', 'Orange', 'Pink', 'Yellow']
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
subColoursList = colours[2:6]
print(subColoursList)
print(colours[2:20])
print(colours[20:2])
print(colours[:5]) #start index is missing
print(colours[2:]) #end index is missing
print(colours[:]) #both indexes are missing
print(colours[:2])
print(colours[2:6:2])
#Negative indexing
print('='*20)
print('Negative Indexing')
print('='*20)
print(colours[-6:-2])
print('='*20)
```

```
print('List in reverse')
print('='*20)
print(colours[::-1])
```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Blue', 'Orange', 'Pink', 'Yellow']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
[]
['Red', 'Green', 'Blue', 'Orange', 'Pink']
['Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
['Red', 'Blue', 'Pink', 'Magenta']
['Blue', 'Pink']
=====
Negative Indexing
=====
['Blue', 'Orange', 'Pink', 'Yellow']
=====
List in reverse
=====
['Cyan', 'Magenta', 'Yellow', 'Pink', 'Orange', 'Blue', 'Green', 'Red']
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
print('colours =', colours)
#Negative indexing
print('='*20)
print('Negative Indexing')
print('='*20)
print(colours[-6:-2])
print('='*20)
print('List in reverse')
print('='*20)
print(colours[::-1])
print('Alternative elements from List in reverse')
print('='*20)
print(colours[::-2])
```

Output:

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
=====
Negative Indexing
=====
['Blue', 'Orange', 'Pink', 'Yellow']
=====
List in reverse
=====
['Cyan', 'Magenta', 'Yellow', 'Pink', 'Orange', 'Blue', 'Green', 'Red']
Alternative elements from List in reverse
=====
['Cyan', 'Yellow', 'Orange', 'Green']
```

Traversing a List

We can access each element of the list or traversal list using a for loop or a while loop.

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
for color in colours:
    print(color)
```

Output:

```
Red
Green
Blue
Orange
Pink
Yellow
Magenta
Cyan
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
for i in range(len(colours)):
    print(colours[i])
```

Output:

```
Red
Green
Blue
Orange
Pink
Yellow
Magenta
Cyan
```

```
colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan']
i = 0
while i < len(colours):
    print(colours[i])
    i += 1
```

Output:

```
Red
Green
Blue
Orange
Pink
Yellow
Magenta
Cyan
```

List methods and built-in functions:

The data type List has several built in methods that are useful in programming.

Built in functions for list manipulations.

Method	Description	Example
len()	Returns the length of the list passed as the argument	<pre>>>> colours = ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan'] >>> len(colours) 8</pre>
list()	Creates an empty list if no argument is passed. Creates a list if a sequence is passed as an argument.	<pre>>>> li = list() >>> li [] >>> str1 = 'aeiou' >>> li1 = list(str1) >>> li1 ['a', 'e', 'i', 'o', 'u']</pre>
append()	Appends a single elements passed as an argument at the end of	<pre>>>> li2 = [10, 20, 30, 40] >>> li2.append(50) >>> li2 [10, 20, 30, 40, 50]</pre>

	the list. The single element can also be a list.	<pre>>>> li2.append([60, 70]) >>> li2 [10, 20, 30, 40, 50, [60, 70]] >>> len(li2) 6</pre>
extends()	Appends each element of the list passed as an argument to the end of the given list.	<pre>>>> li3 = [10,20,30] >>> li4 = [40, 50] >>> li3.extend(li4) >>> li3 [10, 20, 30, 40, 50]</pre>
insert()	Inserts an element at a particular index in the list.	<pre>>>> li3 [10, 20, 30, 40, 50] >>> li3.insert(2,25) >>> li3 [10, 20, 25, 30, 40, 50]</pre>
count()	Returns the number of times a given element appears in the list.	<pre>>>> li = [12,4,5,12,4,4,6,8,5,3,1,10] >>> li.count(4) 3 >>> li.count(5) 2 >>> li.count(20) 0</pre>
index()	Returns a index of the first occurrence of the element in the list. If the element is not present, ValueError is generated.	<pre>>>> li [12, 4, 5, 12, 4, 4, 6, 8, 5, 3, 1, 10] >>> li.index(4) 1 >>> li.index(3) 9 >>> li.index(30) Traceback (most recent call last): File "<stdin>", line 1, in <module> import platform ^^^^^^^^^^^^^^^ ValueError: 30 is not in list</pre>
remove()	Removes the given element from the list. If the element is present multiple times, only the first occurrence is removed. If the element is not present then ValueError is generated.	<pre>>>> li [12, 5, 12, 4, 4, 6, 8, 5, 3, 1, 10] >>> li.remove(20) Traceback (most recent call last): File "<stdin>", line 1, in <module> import platform ^^^^^^^^^^^^^^^ ValueError: list.remove(x): x not in list</pre>

pop()	Returns a element whose index is passed as a parameter to this function and also removes it from the list. If no parameter is given, then it returns and removes the last element of the list.	<pre> >>> li [12, 5, 12, 4, 4, 6, 8, 5, 3, 1, 10] >>> li.pop(3) 4 >>> li [12, 5, 12, 4, 6, 8, 5, 3, 1, 10] >>> li.pop(3) 4 >>> li [12, 5, 12, 6, 8, 5, 3, 1, 10] >>> li.pop() 10 >>> li [12, 5, 12, 6, 8, 5, 3, 1] >>> li.pop(100) Traceback (most recent call last): File "<stdin>", line 1, in <module> import platform ^^^^^^^^^^^ IndexError: pop index out of range >>> li.pop(-100) Traceback (most recent call last): File "<stdin>", line 1, in <module> import platform ^^^^^^^^^^^ IndexError: pop index out of range </pre>
reverse()	Reverses the order of elements in the given list.	<pre> >>> li [12, 5, 12, 6, 8, 5, 3, 1] >>> li.reverse() >>> li [1, 3, 5, 8, 6, 12, 5, 12] >>> colours.reverse() >>> colours ['Cyan', 'Magenta', 'Yellow', 'Pink', 'Orange', 'Blue', 'Green', 'Red'] >>> colours.reverse() >>> colours ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan'] </pre>
sort()	Sort the elements of the given list in-place.	<pre> >>> li [1, 3, 5, 8, 6, 12, 5, 12] >>> li.sort() >>> li [1, 3, 5, 5, 6, 8, 12, 12] >>> colours ['Red', 'Green', 'Blue', 'Orange', 'Pink', 'Yellow', 'Magenta', 'Cyan'] >>> colours.sort() >>> colours ['Blue', 'Cyan', 'Green', 'Magenta', 'Orange', 'Pink', 'Red', 'Yellow'] >>> li = R.sample(range(10, 100), 10) >>> li [56, 50, 28, 75, 82, 32, 33, 46, 10, 41] >>> li.sort(reverse=True) >>> li [82, 75, 56, 50, 46, 41, 33, 32, 28, 10] </pre>
sorted()	It takes a list as a parameter and creates a new list consisting of the same elements arranged in sorted order.	<pre> >>> li = R.sample(range(10, 100), 10) >>> li [27, 63, 38, 53, 99, 71, 90, 18, 45, 36] >>> li2 = sorted(li) >>> li2 [18, 27, 36, 38, 45, 53, 63, 71, 90, 99] >>> li [27, 63, 38, 53, 99, 71, 90, 18, 45, 36] >>> li = R.sample(range(10, 100), 10) >>> li3 = sorted(li, reverse=True) >>> li3 [98, 91, 88, 85, 72, 65, 64, 61, 38, 35] >>> li [61, 38, 91, 85, 35, 88, 65, 72, 64, 98] </pre>
min()	Returns minimum or smallest element of	<pre> >>> li = R.sample(range(10, 100), 10) >>> li [92, 36, 81, 87, 49, 91, 82, 51, 44, 26] </pre>

	the list.	<pre>>>> min(li) 26 >>> max(li) 92 >>> sum(li) 639</pre>
max()	Returns the maximum or largest element of the list.	
sum()	Returns some of the elements of the list.	

Homework

Here are 14 programming questions designed to test the concepts of list slicing and list functions in Python, with varying difficulty levels:

(Simple use of slicing and built-in list functions)

1. Reverse a List Using Slicing

Write a program to reverse a list using list slicing without using the reverse() function.

Example:

Input: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

Solution:

```
li = eval(input('Enter the list elements by separating comma: '))
print(li)
li = li[::-1]
print(li)
```

no space

Output:

```
Enter the list elements by separating comma: [10,20,30,40,50]
[10, 20, 30, 40, 50]
[50, 40, 30, 20, 10]
```

2. Extract Sublist from a List

Given a list of numbers, print the elements from index 2 to 5 (inclusive of index 2, exclusive of index 5).

Example:

Input: [10, 20, 30, 40, 50, 60, 70]

Output: [30, 40, 50]

3. Using append() and insert()

Write a program that takes a list of colors. Add a new color to the end of the list using append(), and insert another color at the second position using insert().

Solution:

```
colours=[]
while True:
    clr = input('Enter colour name: ')
    colours.append(clr)
    done = input('Type done to stop appending: ')
    if done.lower() == 'done':
        break
print(colours)
clr = input('Enter colour name: ')
colours.insert(1, clr)
print(colours)
```

Output:

```
Enter colour name: Brown
Type done to stop appending: c
Enter colour name: Green
Type done to stop appending: c
Enter colour name: Purple
Type done to stop appending: c
Enter colour name: Orange
Type done to stop appending: c
Enter colour name: Yellow
Type done to stop appending: c
Enter colour name: Red
Type done to stop appending: c
Enter colour name: Violet
Type done to stop appending: done
['Brown', 'Green', 'Purple', 'Orange', 'Yellow', 'Red', 'Violet']
Enter colour name: Indigo
['Brown', 'Indigo', 'Green', 'Purple', 'Orange', 'Yellow', 'Red', 'Violet']
```

4. Slicing with Step Value

Write a program to display every second element of a list using slicing.

Example:

Input: [2, 4, 6, 8, 10, 12]

Output: [2, 6, 10]

5. Count Occurrences in a List

Given a list of names, count how many times the name "Alice" appears using the count() function.

(Combining slicing and functions)

1. Find Maximum and Minimum Using List Functions

Write a program to find the largest and smallest number in a list using built-in list functions.

2. Remove Even Numbers Using Slicing

Given a list of numbers, create a new list that contains only the odd numbers using slicing and list comprehension.

Example:

Input: [1, 2, 3, 4, 5, 6]

Output: [1, 3, 5]

3. Using extend() to Merge Lists

Write a program that merges two lists using the extend() function and displays the final list.

4. Replace a Portion of a List Using Slicing

Replace elements from index 2 to 4 with [100, 200, 300] using list slicing.

Example:

Input: [10, 20, 30, 40, 50, 60]

Output: [10, 20, 100, 200, 300, 60]

5. Create a List of Even Indexed Elements

Write a program to create a new list containing only elements at even indexes from the original list using slicing.

(Application-oriented slicing and list operations)

1. List Rotation

Rotate a list to the right by k positions using slicing.

Example:

Input: [1, 2, 3, 4, 5], k = 2

Output: [4, 5, 1, 2, 3]

2. Palindrome Check Using Slicing

Write a program to check if a list is a palindrome using slicing.

Example:

Input: [1, 2, 3, 2, 1]

Output: Palindrome

3. Sort and Slice

Write a program that sorts a list in ascending order and then displays the first five elements using slicing.

Example:

Input: [20, 5, 15, 10, 30, 25]

Output: [5, 10, 15, 20, 25]

4. Nested List - Extract Inner Elements

You are given a nested list. Write a program to extract the first three elements of the second list using slicing.

Example:

Input: [[1, 2, 3], [10, 20, 30, 40, 50], [100, 200]]

Output: [10, 20, 30]

Nested Lists

When a list appears as an element of another list, it is called a nested list.

```
>>> li1 = [1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9]
>>> li1[5]
[6, 7, 8]
>>> li1[5][1]
7
```

To access the element of nested list of li1, we have to specify two indices li1[i][j]. The first index 'i' will take us to the desired nested list and the second index 'j' will take us to the desired element in that nested list.

Copying List

```
>>> li1
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9]
>>> li2 = li1
>>> li2
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9]
>>> li2.append('hello')
>>> li1
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello']
>>>
```

This statement does not create a new list. Rather, it just makes li1 & li2 refer to the same list object. Here li2 actually becomes an alias of li1. Therefore any changes made to either of them will be reflected in the other list.

We can also create a copy or clone of the list as a distinct object by. Three methods. The first method uses slicing, the second method uses built in function list() and the 3rd method uses copy() functions of Python Library.

Method-1

We can slice our original list and store it into a new variable as follows.

```
>>> li3 = li1[:]
>>> li3
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello']
>>> li3.append(100)
>>> li1
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello']
>>> li3
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello', 100]
>>> id(li1)
2196834268032
>>> id(li2)
2196834268032
>>> id(li3)
2196826908096
```

Method- 2

We can use the built in function list () as follows:

```
>>> li4 = list(li2)
>>> id(li4)
2196834461952
>>> id(li2)
2196834268032
>>> li4
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello']
>>> li4.append(67)
>>> li4
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello', 67]
>>> li2
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello']
```

Method 3

We can use the copy () Function as follows:

```
>>> import copy
>>> li5 = copy.copy(li4)
>>> id(li5)
2196834442368
>>> id(li4)
2196834461952
>>> print(li5)
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello', 67]
>>> li5.pop(5)
[6, 7, 8]
>>> li5
[1, 2, 3, 'a', 'c', 4, 9, 'hello', 67]
>>> li4
[1, 2, 3, 'a', 'c', [6, 7, 8], 4, 9, 'hello', 67]
```